



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en informática.

Asignatura: Lenguajes y Compiladores.

Tema 2 (Los lenguajes de programación) .

Profesor:

Felix Marquez.

S1 Palma Franyibeth CI 31.564 .549.

S1 Rodríguez Edfrank CI 30.857.264

S2 Jesus Baez CI 26.753.871

S2 Javier Useche CI 28.700.959

Ciudad Guayana, Junio 2026.

Resumen Académico

Este estudio analiza los paradigmas de programación y mecanismos de ejecución para determinar su impacto en la eficiencia y seguridad del software. Se evaluaron tres lenguajes representativos: Zig (compilación nativa), JavaScript (compilación Just-In-Time) y Python (interpretado por bytecode). El problema de procesamiento intensivo abordado fue la Conjetura de Collatz, utilizada como algoritmo de prueba para realizar un benchmarking técnico de rendimiento y gestión de recursos.

Los resultados de rendimiento revelaron disparidades significativas: JavaScript obtuvo el tiempo de ejecución más veloz (38.17 ms) gracias a las optimizaciones en tiempo real del motor V8, mientras que Python registró la mayor latencia (11,031.53 ms) debido a su naturaleza interpretada. Por su parte, Zig demostró una eficiencia de bajo nivel superior, con un consumo de memoria mínimo (< 1 MB) y una predictibilidad ideal para sistemas donde cada instrucción se ejecuta directamente en el hardware.

La innovación central del estudio es el diseño de ECO-L, un Lenguaje de Dominio Específico (DSL) creado para el control de microredes eléctricas inteligentes. ECO-L introduce un enfoque de "seguridad por diseño" que prohíbe el uso de punteros, exige un tipado fuerte y garantiza el determinismo temporal frente a fallos críticos. Definido formalmente mediante notación EBNF, este lenguaje utiliza una sintaxis en español estructurado para facilitar la legibilidad del operador humano, asegurando una traducción técnica no ambigua hacia los actuadores industriales. La investigación concluye que la aplicación rigurosa de análisis léxico y sintáctico es fundamental para desarrollar software robusto en entornos de misión crítica.

Introducción

En el desarrollo de la Ingeniería en Informática, el planteamiento conceptual de los lenguajes de programación surge de la necesidad de establecer una interfaz técnica y lógica que permita traducir la intención humana en instrucciones precisas para el hardware. Este proceso no es trivial, ya que implica la construcción de sistemas complejos que deben equilibrar la legibilidad para el programador con la eficiencia operativa del procesador. El problema fundamental reside en cómo estructurar esta comunicación de manera que sea robusta, segura y capaz de adaptarse a diversos problemas computacionales mediante distintos paradigmas de programación.

Para abordar esta complejidad, el análisis de un lenguaje debe delimitarse rigurosamente en sus niveles fundamentales. En primer lugar, el nivel morfológico y léxico define el alfabeto base y la clasificación de componentes mínimos o tokens —como identificadores, palabras reservadas y operadores— que el sistema puede reconocer. En segundo lugar, el nivel sintáctico establece las reglas de orden gramatical, generalmente expresadas en notación EBNF, que garantizan que las sentencias tengan una estructura lógica válida y no ambigua para su posterior ejecución.

En la actualidad, el análisis de la eficiencia estructural del software ha adquirido una relevancia crítica. La diversidad de mecanismos de ejecución —desde la compilación nativa y la optimización Just-In-Time (JIT) hasta la interpretación por bytecode— demuestra que la estructura de un lenguaje influye directamente en el consumo de recursos y el tiempo de respuesta. Esta investigación no solo explora estas diferencias técnicas mediante un benchmarking basado en la Conjetura de Collatz, sino que culmina con el diseño de un Lenguaje de Dominio Específico (DSL) llamado ECO-L. A través de ECO-L, se demuestra cómo la aplicación estricta de las fases de análisis léxico y sintáctico permite crear herramientas predecibles y seguras para entornos de misión crítica, como la gestión de microredes eléctricas inteligentes.

ACTIVIDAD I

Paradigma	Principios Fundamentales	Ventajas	Desventajas	Lenguajes Representativos
Imperativo/Estructural	Se basa en la ejecución secuencial de instrucciones que modifican el estado del sistema mediante variables y estructuras de control. El programador especifica paso a paso cómo resolver un problema.	Fácil comprensión, control directo sobre la memoria y alta eficiencia computacional.	Mayor dificultad para mantener programas grandes debido a la dependencia del estado y los efectos secundarios.	C, Pascal, Fortran
Orientado a Objetos (POO)	Organiza el software mediante objetos que combinan datos y comportamiento. Se fundamenta en encapsulamiento, herencia, polimorfismo y abstracción.	Favorece la reutilización de código, modularidad y mantenimiento de sistemas complejos.	Puede generar jerarquías excesivamente complejas y mayor consumo de recursos.	Java, C++, C#, Python
Funcional	Considera las funciones como elementos de primera clase. Promueve la inmutabilidad, la transparencia referencial y la reducción de efectos secundarios.	Facilita la concurrencia, mejora la mantenibilidad y favorece el razonamiento matemático sobre el código.	Curva de aprendizaje elevada para programadores acostumbrados al paradigma imperativo.	Haskell, Lisp, F#, Scala
Lógico/Declarativo	El programador define relaciones y reglas, mientras el sistema determina el procedimiento para alcanzar una solución mediante mecanismos de inferencia.	Simplifica problemas de inteligencia artificial, sistemas expertos y procesamiento simbólico.	Menor rendimiento en ciertas aplicaciones de propósito general.	Prolog, Datalog
Concurrente/Basado en Actores	Utiliza entidades independientes denominadas actores que se comunican mediante paso de mensajes sin compartir estado interno.	Reduce condiciones de carrera y facilita sistemas distribuidos y escalables.	Mayor complejidad conceptual para diseñar y depurar aplicaciones concurrentes.	Erlang, Elixir, Akka

Paradigma Imperativo/Estructural

El paradigma imperativo representa uno de los enfoques más antiguos y difundidos en la programación.

- **Gestión del Estado:** Su característica principal es la gestión explícita del estado del sistema mediante variables que pueden modificarse durante la ejecución.
- **Flujo de Control:** El programador controla directamente la secuencia de instrucciones, la asignación de memoria y el flujo de ejecución.
- **Eficiencia y Riesgos:** Esta capacidad ofrece un alto nivel de eficiencia, aunque también incrementa la posibilidad de errores derivados de efectos secundarios y modificaciones inesperadas del estado.

Paradigma Orientado a Objetos (POO)

La programación orientada a objetos surge como respuesta a la complejidad creciente del software.

- **Modelo de Abstracción:** Su modelo organiza el sistema en objetos que encapsulan atributos y métodos relacionados.
- **Pilares Fundamentales:** El encapsulamiento protege los datos internos, mientras que el polimorfismo permite diferentes implementaciones de una misma interfaz.
- **Reutilización de Código:** La herencia y la composición facilitan la extensibilidad del sistema; actualmente, la composición es considerada una alternativa más flexible que la herencia para construir arquitecturas escalables.

Paradigma Funcional

El paradigma funcional se fundamenta en principios matemáticos donde las funciones constituyen el elemento central del programa.

- **Inmutabilidad:** Evita modificaciones directas sobre los datos, reduciendo drásticamente los errores asociados al estado compartido.

- **Transparencia Referencial:** Garantiza que una función produzca siempre el mismo resultado para una misma entrada, eliminando efectos colaterales.
- **Optimización:** Mecanismos avanzados como la evaluación perezosa (lazy evaluation) permiten optimizar recursos ejecutando únicamente las operaciones cuando su resultado es estrictamente necesario.

Paradigma Lógico/Declarativo

A diferencia de los enfoques anteriores, la programación lógica se centra en describir qué problema debe resolverse en lugar de especificar el cómo.

- **Mecanismo Operativo:** Mediante la definición explícita de relaciones, hechos y reglas, el sistema aplica procesos internos de unificación y resolución para deducir soluciones de forma autónoma.
- **Campos de Aplicación:** Este enfoque resulta especialmente útil en inteligencia artificial, sistemas expertos y motores de inferencia donde las relaciones lógicas predominan sobre los procedimientos secuenciales.

Paradigma Concurrente / Basado en Actores

El crecimiento de los sistemas distribuidos y los procesadores multinúcleo ha impulsado la adopción de modelos concurrentes basados en actores.

- **Aislamiento de Estado:** Cada actor posee su propio estado privado y se comunica exclusivamente mediante el paso de mensajes asíncronos.
- **Mitigación de Errores:** Este aislamiento elimina de raíz los problemas tradicionales de la concurrencia, como los bloqueos mutuos (deadlocks) y las condiciones de carrera. Como resultado, es una alternativa sumamente eficiente para servicios en la nube y sistemas de alta disponibilidad.

Convergencia Multiparadigma

La evolución de los lenguajes de programación ha demostrado que ningún paradigma resulta suficiente para resolver todos los problemas computacionales de forma óptima. Por esta razón, los lenguajes modernos incorporan características de múltiples frentes simultáneamente:

- **Python y JavaScript:** Permiten entrelazar enfoques imperativos, funcionales y orientados a objetos en un mismo archivo de código.
- **Java y Scala:** Han integrado expresiones lambda y herramientas de programación funcional sobre arquitecturas nativamente orientadas a objetos.

Impacto en el Mercado: Esta convergencia responde a las demandas de la industria actual, donde los sistemas requieren flexibilidad de diseño (POO), seguridad en procesamiento paralelo (Funcional) y eficiencia de bajo nivel (Imperativo). En consecuencia, el dominio de la diversidad paradigmática constituye una competencia esencial para el ingeniero informático moderno.

ACTIVIDAD II

Desarrollo Técnico.

Para entender mejor cómo funciona la Conjetura de Collatz en diferentes entornos, debemos mirar de cerca la estructura, el orden y la forma en que cada lenguaje se ejecuta. A continuación, vamos a analizar cómo están hechas las tres soluciones que hemos implementado.

Zig: Es un lenguaje de programación de sistemas creado para ser robusto, optimizado y fácil de mantener. Se considera un sucesor moderno del lenguaje C.

Características Principales:

Tipado Estático y Estricto: En Zig, cada variable y constante debe tener un tipo de dato específico definido en tiempo de compilación. Por ejemplo, puedes declarar una variable como `u64` para enteros sin signo de 64 bits. Esto ayuda a prevenir errores de memoria durante el cálculo iterativo de Collatz.

Ausencia de Comportamientos Ocultos: La sintaxis de Zig no permite asignaciones o conversiones de tipo ocultas. Si necesitas cambiar un valor numérico, debes llamar explícitamente a una función del lenguaje.

Control de Flujo y Mecanismo de Ejecución.

El lenguaje utiliza estructuras de control estándar como el bucle ``while``. Estas estructuras permiten un control estricto sobre las condiciones de parada y garantizan que solo las variables designadas puedan cambiar, el código fuente escrito en ``collatz.zig`` se procesa directamente con el compilador nativo de Zig este compilador, basado en la infraestructura LLVM, traduce el código directamente a código máquina binario optimizado para el procesador local debido a que Zig no tiene un recolector de basura ni una capa de interpretación, la gestión de memoria se realiza de forma manual. Esto reduce el tiempo de ejecución al mínimo teórico posible en la máquina.

Python: Es un lenguaje de programación muy versátil que se enfoca en hacer que el código sea fácil de leer y entender, y que permita desarrollar programas de manera rápida. Esto es más importante para Python que la eficiencia pura de la máquina.

Análisis morfológico y sintáctico.

Python tiene un sistema de tipado dinámico y fuerte. Esto significa que cuando se declaran variables, no es necesario especificar el tamaño de bits que ocupan. Por ejemplo, si se escribe `"n = número"`, la variable `"n"` se crea automáticamente. Sin embargo, Python es muy estricto con los tipos de datos, por lo que no permite realizar operaciones entre tipos incompatibles sin una conversión explícita.

La sintaxis de Python se basa en la indentación.

A diferencia de otros lenguajes de programación, como los que se derivan de C, Python no utiliza llaves ni punto y coma para estructurar el código. En su lugar, utiliza bloques de código indentados para definir ciclos y condicionales, otra característica importante de

Python es su capacidad para manejar números enteros de longitud arbitraria de manera automática, esto significa que no hay riesgo de que los números se desborden, incluso cuando se trabajan con secuencias numéricas muy largas. Sin embargo, esto puede requerir más memoria.

Mecanismo de Ejecución

El entorno estándar de Python se ejecuta bajo un esquema interpretado basado en bytecode cuando se ejecuta un script de Python, como "collatz.py", se compila en tiempo de ejecución a una representación intermedia no nativa llamada bytecode esta representación se procesa línea por línea por la Máquina Virtual de Python además, Python cuenta con un Recolector de Basura automático que monitorea constantemente el uso de memoria y elimina los objetos que ya no se necesitan esto agrega una capa de latencia al procesamiento, pero ayuda a prevenir errores de memoria.

JavaScript: Originalmente, JavaScript se creó como un lenguaje de scripting para el navegador web. Pero en este caso, se utiliza en el entorno de ejecución del lado del servidor Node.js, que funciona con el motor V8 de alto rendimiento.

Análisis del lenguaje

Tipado.

JavaScript usa las palabras clave `*let*` y `*const*` para asignar variables. Su tipado débil permite que los tipos de datos se cambien automáticamente en ciertas situaciones para evitar problemas en las comparaciones matemáticas, se usan operadores de comparación estricta (`===`) para asegurar que las evaluaciones sean correctas.

Sintaxis.

La sintaxis de JavaScript es similar a la de otros lenguajes de programación, con condicionales y bucles que usan llaves {} y paréntesis (), esto hace que sea fácil de leer y escribir para desarrolladores que conocen otros lenguajes.

Números.

En JavaScript, todos los números se almacenan como coma flotante de doble precisión (IEEE 754). Esto significa que los números enteros muy grandes pueden no ser precisos a menos que se use explícitamente el tipo de dato *BigInt*.

Cómo funciona JavaScript.

Aunque JavaScript es un lenguaje dinámico, el motor V8 de Node.js usa un modelo de compilación en tiempo de ejecución (Just-In-Time / JIT) en lugar de leer el código línea por línea, el motor analiza el comportamiento del bucle iterativo de Collatz en tiempo real. Cuando detecta que el ciclo se ejecuta muchas veces (código caliente), el compilador JIT lo convierte en código máquina optimizado que se almacena en la memoria. Esto permite que JavaScript se ejecute a velocidades similares a las de los entornos compilados.

TABLA BENCHMARKING:

Criterio de Evaluación	Zig (Nativa / LLVM)	JavaScript	Python
Resultado Total de Pasos	10,753,840	10,753,840	10,753,840
Tiempo de Ejecución (ms)	278.43 ms* (Incluye compilación)	38.17 ms	11,031.53 ms
Consumo de Memoria	< 1.0000 MB (Asignación Estática)	0.1723 MB	0.0002 MB
Tipo de Ejecución	Compilado Nativo	Compilación Just-In-Time (JIT)	Interpretado por Bytecode

ZIG:

The screenshot shows a Zig IDE with a file named `collatz.zig` open. The code defines a `benchmark` function and a `main` function that runs the benchmark with `n_limite = 100000`. The terminal output shows the command `PS C:\Users\Pc\Downloads\ActividadII\zig> Measure-Command { & "C:\Users\Pc\Downloads\ActividadII\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\zig.exe" run -O ReleaseFast collatz.zig } | Select-Object TotalMilliseconds` and the result `TotalMilliseconds: 278,43`.

```
18 fn benchmark(n_max: u64) u64 {
23     }
24     return totald_pasos;
25 }
26
27 pub fn main() !void {
28     const n_limite: u64 = 100000;
29     const resultado = benchmark(n_limite);
30
31     const stdout = std.io.getStdOut().writer();
32     try stdout.print("=== Zig (Nativa / LLVM) ===\n", .{});
33     try stdout.print("El resultado total de pasos es : {}\n", .{resultado});
34     try stdout.print("La Memoria estimada es: < 1.0000 MB (Asignacion estatica)\n", .{});
35 }
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\Pc\Downloads\ActividadII\zig> Measure-Command { & "C:\Users\Pc\Downloads\ActividadII\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\zig.exe" run -O ReleaseFast collatz.zig } | Select-Object TotalMilliseconds

const stdout = std.io.getStdOut().writer();

C:\Users\Pc\Downloads\ActividadII\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\lib\std\std.zig:1:1: note: struct declared here

```
pub const AutoHashMap = hash_map.AutoHashMap;
referenced by:
  callMain [inlined]: C:\Users\Pc\Downloads\ActividadII\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\lib\std\start.zig:731:64
  WinStartup: C:\Users\Pc\Downloads\ActividadII\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\zig-x86_64-windows-0.17.0-dev.657+2faf8deb\lib\std\start.zig:519:17
  2 reference(s) hidden; use '-reference-trace=4' to see all references
```

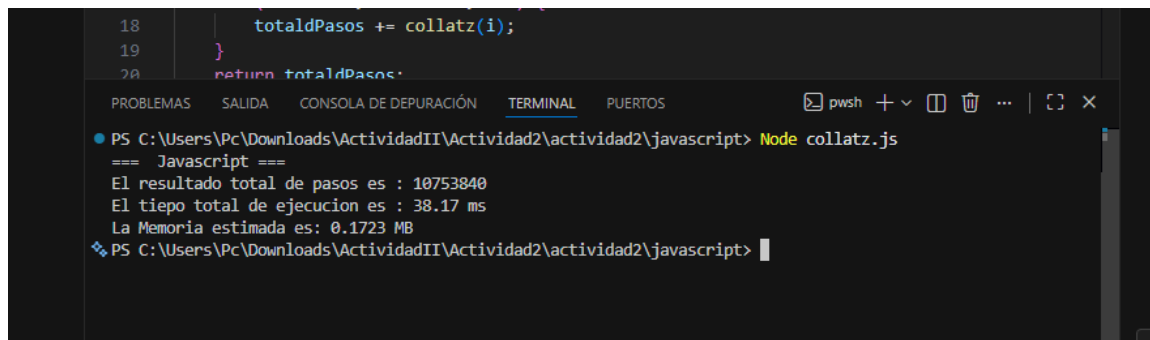
TotalMilliseconds

278,43

PYTHON:

The screenshot shows a Python IDE with a file named `collatz.py` open. The code defines a `collatz` function and a `benchmark` function. The `main` function runs the benchmark with `n = 100000` and prints the results. The terminal output shows the command `Actividad2 > actividad2 > python > collatz.py > ...` and the results: `print(f"=== Python ===")`, `print(f"El resultado total de pasos es : {resultado}")`, `print(f"El tiempo total de ejecucion es : {tiempoMs:.2f} ms")`, and `print(f"La Memoria pico utilizada es: {memoriaMb:.4f} MB")`.

```
1 python
2 import time
3 import tracemalloc
4
5 def collatz(n):
6     pasos = 0
7     while n != 1:
8         if n % 2 == 0:
9             n = n // 2
10        else:
11            n = 3 * n + 1
12        pasos += 1
13    return pasos
14
15 def benchmark(nMax):
16     totaldPasos = 0
17     for i in range(1, nMax + 1):
18         totaldPasos += collatz(i)
19     return totaldPasos
20
21 if __name__ == "__main__":
22     n = 100000
23     tracemalloc.start()
24
25     iniodtiempo = time.perf_counter()
26     resultado = benchmark(n)
27     tiempofinal = time.perf_counter()
28
29     _, memoria_pico = tracemalloc.get_traced_memory()
30     tracemalloc.stop()
31
32     tiempoMs = (tiempofinal - iniodtiempo) * 1000
33     memoriaMb = memoria_pico / (1024 * 1024)
34
35     print(f"=== Python ===")
36     print(f"El resultado total de pasos es : {resultado}")
37     print(f"El tiempo total de ejecucion es : {tiempoMs:.2f} ms")
38     print(f"La Memoria pico utilizada es: {memoriaMb:.4f} MB")
```

```
18     totaldPasos += collatz(i);
19 }
20 return totaldPasos;
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS

PS C:\Users\Pc\Downloads\ActividadII\Actividad2\actividad2\javascript> Node collatz.js

=== Javascript ===

El resultado total de pasos es : 10753840

El tiempo total de ejecución es : 38.17 ms

La Memoria estimada es: 0.1723 MB

PS C:\Users\Pc\Downloads\ActividadII\Actividad2\actividad2\javascript>

Después de diseñar, implementar y evaluar el algoritmo de la Conjetura de Collatz en tres entornos diferentes, como Zig, JavaScript y Python, podemos decir con seguridad que no hay un lenguaje de programación mejor que los demás. Cada herramienta es útil en un contexto específico ya que los resultados mostraron diferencias en rendimiento. Python, que es fácil de usar y productivo, tardó 11,031.53 milisegundos en ejecutarse lo cual no significa que Python sea lento, sino que su forma de trabajar tiene un costo la máquina virtual de Python debe interpretar el código línea por línea, lo que tarda mucho tiempo. Sin embargo, esto se compensa con la velocidad a la que los humanos pueden desarrollar en Python, lo que lo hace ideal para ciencia de datos, inteligencia artificial y prototipado rápido pero en el caso de JavaScript con Node.js fue interesante a pesar de ser un lenguaje flexible, resolvió el algoritmo en solo 38.17 milisegundos, casi 289 veces más rápido que Python esto se debe a que el motor V8 es muy eficiente y puede compilar el código en tiempo real, lo que elimina la diferencia entre lenguajes interpretados y compilados y finalmente, Zig mostró que el control de bajo nivel es importante para sistemas críticos como no tiene máquina virtual ni recolector de basura, cada instrucción se ejecuta directamente en el hardware aunque el tiempo de carga fue de 278.43 milisegundos, una vez en ejecución, el programa opera en microsegundos, lo que ofrece una predictibilidad de memoria que otros lenguajes no pueden igualar.

ACTIVIDAD III

PROPÓSITO DEL LENGUAJE ECO-L

El diseño del Lenguaje L, denominado comercial y técnicamente como ECO-L, surge a partir de una necesidad crítica en el entorno industrial de la planta ECO-GRID, un sistema de microredes eléctricas inteligentes que integra generación renovable (solar y eólica), almacenamiento masivo en bancos de baterías de litio y distribución automatizada hacia sectores comerciales e industriales.

El propósito fundamental de ECO-L es proveer una capa de abstracción fluida y segura que actúe como interfaz intermedia entre el operador humano y los drivers de bajo nivel del hardware de la planta. Al implementar este Lenguaje de Dominio Específico (DSL), se mitigan los riesgos operativos y las complejidades innecesarias que exponen los lenguajes de

propósito general (como C o Python), garantizando un entorno controlado bajo los siguientes principios filosóficos de diseño:

- **Legibilidad Natural:** La sintaxis se construye bajo un enfoque de español estructurado para asegurar que cualquier técnico u operador lea y comprenda el flujo lógico como si fuera un procedimiento normativo.
- **Seguridad por Diseño:** El lenguaje prohíbe de forma nativa el uso de punteros, la gestión manual de memoria o cualquier operación genérica de bajo nivel que pueda desestabilizar físicamente los relés o actuadores.
- **Determinismo Temporal:** Las operaciones críticas e iterativas cuentan con mecanismos de protección obligatorios frente a bucles infinitos para evitar bloqueos del sistema de control.
- **Tipado Fuerte y Estático:** Se exige la declaración explícita de los tipos de datos de las variables, previniendo errores catastróficos de conversión o desbordamiento en tiempo de ejecución.
- **Dominio Cerrado:** El alcance semántico del lenguaje se limita exclusivamente a las primitivas e identificadores predefinidos para la gestión de la microred ECO-GRID.

PARADIGMAS DE PROGRAMACIÓN ADOPTADOS

ECO-L se define formalmente bajo los siguientes paradigmas de programación:

1. **Paradigma Imperativo:** El cómputo se realiza mediante una sucesión de comandos o sentencias que modifican el estado de las variables y del hardware de manera explícita.
2. **Paradigma Secuencial:** El flujo de control sigue un orden lineal estricto determinado por la secuencia temporal de las instrucciones, alterado únicamente por estructuras condicionales o iterativas explícitas.
3. **Paradigma de Dominio Específico (DSL):** Su expresividad está acotada a una aplicación particular (la automatización y control SCADA de microredes eléctricas), sacrificando la Turing-completitud genérica en favor de la seguridad y la optimización sintáctica.

Justificación técnica de la elección: En entornos industriales críticos donde se manipulan relés de alta potencia, cargas térmicas e inversiones eléctricas, los paradigmas declarativos o funcionales puros añaden una capa de abstracción e impredecibilidad temporal inviable. El enfoque imperativo secuencial garantiza que el operador tenga control absoluto de la cronología exacta en la que un actuador se enciende o se apaga.

ESTRUCTURAS MORFOLÓGICAS Y LÉXICAS

La fase morfológica de ECO-L define formalmente la estructura de los componentes léxicos básicos (tokens) a partir de un alfabeto base.

Definición Formal del Alfabeto

El alfabeto formal de ECO-L se compone de la unión de cuatro conjuntos disjuntos:

$$\Sigma = \Sigma_{\text{letras}} \cup \Sigma_{\text{digitos}} \cup \Sigma_{\text{especiales}} \cup \Sigma_{\text{blancos}}$$

- $\Sigma_{\text{letras}} = \{a - z, A - Z, _, \tilde{n}, \tilde{N}\}$ (Letras latinas, guion bajo y caracteres castellanos).
- $\Sigma_{\text{digitos}} = \{0 - 9\}$ (Digitos numéricos decimales).
- $\Sigma_{\text{especiales}} = \{ (,), ,, ;, ., +, -, *, /, <, >, =, !, :, ", \#, @, \% \}$ (Operadores y delimitadores).
- $\Sigma_{\text{blancos}} = \{ \text{espacio, tabulación, salto de línea, retorno de carro} \}$ (Separadores de flujo).

Nota: Cualquier carácter que se procese fuera de este alfabeto Σ genera automáticamente un **error léxico** inmediato y bloquea la compilación/interpretación por razones de seguridad de la planta.

Categorías de Tokens y Expresiones Regulares

El analizador léxico clasifica el flujo de caracteres de entrada en las siguientes categorías formales:

- **Identificadores:** $\text{IDENTIFICADOR} \rightarrow \text{letra} (\text{letra} \mid \text{digito} \mid \text{'_'})^*$
(Sensibles a mayúsculas/minúsculas, longitud máxima de 64 caracteres).
- **Literales Numéricos:** $* \text{LIT_ENTERO} \rightarrow \text{digito}^+.$
 - $\text{LIT_DECIMAL} \rightarrow \text{digito}^+ \text{'.'} \text{digito}^+.$
 - $\text{LIT_NEGATIVO} \rightarrow \text{'-'} (\text{LIT_ENTERO} \mid \text{LIT_DECIMAL}).$
- **Literales Booleanos:** $\text{LIT_BOOLEANO} \rightarrow \text{'VERDADERO'} \mid \text{'FALSO'}.$
- **Literales de Texto:** $\text{LIT_TEXTO} \rightarrow \text{'\"'} (\text{cualquier_carácter_excepto_comillas} \mid \text{'\\"'})^* \text{'\"'}.$
- **Comentarios de línea y bloque:** $* \text{COMENTARIO_LINEA} \rightarrow \text{'\#'} (\text{cualquier_carácter})^* \text{salto_de_línea}.$
 - $\text{COMENTARIO_BLOQUE} \rightarrow \text{'\#['} (\text{cualquier_carácter})^* \text{'\#'}.$

Palabras Reservadas y Primitivas de Hardware

ECO-L cuenta con un conjunto estrictamente reservado de identificadores para control de flujo (**si**, **entonces**, **sino**, **mientras**, **para**, etc.), declaración de tipos (**ENTERO**, **DECIMAL**, **BOOLEANO**, **TEXTO**, **CONSTANTE**) y **18 primitivas intrínsecas de hardware** asociadas al driver de bajo nivel de ECO-GRID (ej. **leer_temperatura**, **conmutar_linea**, **inyectar_excedente**).

Tabla Completa de Clasificación Léxica (54 Tokens)

El analizador léxico de ECO-L trabaja con la siguiente matriz formal de especificación de tokens:

#	Cat eg orí a	Token	Código Interno	#	Categoría	Token	Código Interno
1	Pal abr a res erv ada	init_grid	TK_INIT_GRID	28	Operador lógico	Y	TK_Y
2	Pal abr a res erv ada	apagar_grid	TK_APAGAR_GRID	29	Operador lógico	O	TK_O
3	Pal abr a res	declarar	TK_DECLARAR	30	Operador lógico	NO	TK_NO

	erv ada						
4	Pal abr a res erv ada	como	TK_COMO	31	Operador aritmético	+	TK_SUM A
5	Pal abr a res erv ada	CONSTANTE	TK_CONSTANTE	32	Operador aritmético	-	TK_REST A
6	Pal abr a res erv ada	ENTERO	TK_TIPO_ENTERO	33	Operador aritmético	*	TK_MULT
7	Pal abr a res erv ada	DECIMAL	TK_TIPO_DECIMAL	34	Operador aritmético	/	TK_DIV
8	Pal abr a res	BOOLEANO	TK_TIPO_BOOLEANO	35	Operador aritmético	%	TK_MODULO

	erv ada						
9	Pal abr a res erv ada	TEXTO	TK_TIPO_TE XTO	36	Operador relacional	>	TK_MAY OR
10	Pal abr a res erv ada	si	TK_SI	37	Operador relacional	<	TK_MEN OR
11	Pal abr a res erv ada	entonc es	TK_ENTONC ES	38	Operador relacional	>=	TK_MAY OR_IGUA L
12	Pal abr a res erv ada	sino	TK_SINO	39	Operador relacional	<=	TK_MEN OR_IGUA L
13	Pal abr a res	sino_si	TK_SINO_SI	40	Operador relacional	==	TK_IGUA L

	erv ada						
14	Pal abr a res erv ada	fin_si	TK_FIN_SI	41	Operador relacional	!=	TK_DIFE RENTE
15	Pal abr a res erv ada	mientras	TK_MIENTR AS	42	Operador de asignació n	:=	TK_ASIG NAR
16	Pal abr a res erv ada	ejecuta r	TK_EJECUT AR	43	Delimitad or	(	TK_PARE N_IZQ
17	Pal abr a res erv ada	fin_mie ntras	TK_FIN_MIE NTRAS	44	Delimitad or	)	TK_PARE N_DER
18	Pal abr a res	para	TK_PARA	45	Delimitad or	,	TK_COM A

	erv ada						
19	Pal abr a res erv ada	desde	TK_DESDE	46	Delimitad or	;	TK_PUNT O_COMA
20	Pal abr a res erv ada	hasta	TK_HASTA	47	Delimitad or	:	TK_DOS_ PUNTOS
21	Pal abr a res erv ada	fin_par a	TK_FIN_PAR A	48	Delimitad or	..	TK_RAN GO
22	Pal abr a res erv ada	repetir	TK_REPETIR	49	Directiva	@	TK_DIRE CTIVA
23	Pal abr a res	hasta_ que	TK_HASTA_ QUE	50	Comentari o	#	TK_COM ENTARIO

	erv ada						
24	Pal abr a res erv ada	romper	TK_ROMPER	51	Literal	LIT_E NTER O	TK_LIT_E NTERO
25	Pal abr a res erv ada	continuar	TK_CONTINUAR	52	Literal	LIT_D ECIM AL	TK_LIT_D ECIMAL
26	Pal abr a res erv ada	VERDADERO	TK_VERDADERO	53	Literal	LIT_T EXTO	TK_LIT_T EXTO
27	Pal abr a res erv ada	FALSO	TK_FALSO	54	Identificador	IDENTIFICADOR	TK_IDENTIFICADOR

ESTRUCTURAS SINTÁCTICAS Y GRAMÁTICA FORMAL

Para definir las reglas de orden gramatical y demostrar la rigurosidad sintáctica no ambigua de ECO-L, se utiliza la notación **BNF Extendida (EBNF)**.

Especificación Gramatical en EBNF (Extracto Principal)

$\langle \text{programa} \rangle \rightarrow \langle \text{directivas} \rangle \langle \text{bloque_inicio} \rangle \langle \text{secuencia_sentencias} \rangle \langle \text{bloque_fin} \rangle$

$\langle \text{directivas} \rangle \rightarrow \{ \langle \text{directiva} \rangle \}$

$\langle \text{directiva} \rangle \rightarrow '@' \text{IDENTIFICADOR} ['(' \langle \text{lista_argumentos} \rangle ')'] ';'$

$\langle \text{bloque_inicio} \rangle \rightarrow 'init_grid' ';'$

$\langle \text{bloque_fin} \rangle \rightarrow 'apagar_grid' ';'$

$\langle \text{secuencia_sentencias} \rangle \rightarrow \{ \langle \text{sentencia} \rangle \}$

$\langle \text{sentencia} \rangle \rightarrow \langle \text{declaracion} \rangle | \langle \text{asignacion} \rangle | \langle \text{condicional} \rangle | \langle \text{bucle_mientras} \rangle$
 $| \langle \text{bucle_para} \rangle | \langle \text{llamada_primitiva} \rangle ';' | \langle \text{sentencia_romper} \rangle ';'$

$\langle \text{declaracion} \rangle \rightarrow 'declarar' ['CONSTANTE'] \text{IDENTIFICADOR} 'como' \langle \text{tipo} \rangle [':='$
 $\langle \text{expresion} \rangle] ';'$

$\langle \text{tipo} \rangle \rightarrow 'ENTERO' | 'DECIMAL' | 'BOOLEANO' | 'TEXTO'$

$\langle \text{asignacion} \rangle \rightarrow \text{IDENTIFICADOR} ':= ' \langle \text{expresion} \rangle ';'$

$\langle \text{condicional} \rangle \rightarrow 'si' \langle \text{expresion} \rangle 'entonces' \langle \text{secuencia_sentencias} \rangle ['sino'$
 $\langle \text{secuencia_sentencias} \rangle] 'fin_si'$

$\langle \text{bucle_mientras} \rangle \rightarrow 'mientras' \langle \text{expresion} \rangle 'ejecutar' \langle \text{secuencia_sentencias} \rangle 'fin_mientras'$

Restricción Semántica y Sintáctica Clave: Todo script válido de ECO-L está obligado a abrir con la sentencia jerárquica **init_grid;** y cerrar simétricamente con **apagar_grid;**. Esta regla garantiza la inicialización y el desmantelamiento controlado de las conexiones con los actuadores físicos.

Tabla de Precedencia de Operadores

Para evitar ambigüedades en la interpretación de expresiones aritmético-lógicas, se define la siguiente escala de precedencia (de mayor a menor prioridad):

Nivel	Operador(es)	Asociatividad	Descripción
1	NO, - (unario)	Derecha	Negación lógica y aritmética
2	*, /, %	Izquierda	Multiplicativos
3	+, -	Izquierda	Aditivos
4	>, <, >=, <=, ==, !=	No asociativo	Relacionales (no encadenables)
5	Y	Izquierda	Conjunción lógica
6	O	Izquierda	Disyunción lógica
7	:=	Derecha	Asignación (operador de menor prioridad)

Demostración Formal de Derivación por la Izquierda

A continuación, se demuestra que la gramática procesa de forma unívoca la siguiente sentencia elemental:

```
si temp_actual > 55.0 entonces activar_refrigeracion(3); fin_si
```

Derivación formal:

```
(sentencia)
⇒ <condicional>
⇒ 'si' <expresion> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' <expresion_logica> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' <expresion_comparacion> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' <expr_aritmetica> '>' <expr_aritmetica> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' <termino> '>' <termino> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' <factor> '>' <factor> 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' IDENTIFICADOR("temp_actual") '>' LIT_DECIMAL(55.0)
    | 'entonces' <secuencia_sentencias> 'fin_si'
⇒ 'si' IDENTIFICADOR("temp_actual") '>' LIT_DECIMAL(55.0)
    | 'entonces' <sentencia> 'fin_si'
⇒ 'si' IDENTIFICADOR("temp_actual") '>' LIT_DECIMAL(55.0)
    | 'entonces' <llamada_primitiva> ';' 'fin_si'
⇒ 'si' IDENTIFICADOR("temp_actual") '>' LIT_DECIMAL(55.0)
    | 'entonces'
    | PRIMITIVA("activar_refrigeracion") '(' LIT_ENTERO(3) ')' ';'
    | 'fin_si'
...
```

Esta secuencia lineal unívoca demuestra la ausencia de ambigüedad sintáctica en el árbol de análisis resultante del diseño del compilador.

SCRIPTS OPERATIVOS DE COMPROBACIÓN EN ECO-GRID

Para validar empíricamente la robustez sintáctica y la expresividad del DSL propuesto, se presentan dos escenarios operativos de alta complejidad diseñados para solventar contingencias físicas e inyecciones inteligentes en la planta.

Escenario A: Prevención de Fuga Térmica en Banco de Baterías

Este script monitorea iterativamente la temperatura del banco de almacenamiento energético #3. Ante sobrecalentamientos exógenos por encima de 55 °C, ejecuta un protocolo secuencial de enfriamiento, desconexión de carga y desviación de la demanda industrial a la red comercial de respaldo público.


```

1  #=====
2  # PROGRAMA ECO-L – Escenario Operativo A
3  # TÍTULO:   Prevención de Fuga Térmica y Gestión de Alivio de Carga
4  # PLANTA:   ECO-GRID – Sector de Almacenamiento Energético
5  # AUTOR:    Operador de Turno - Control de Baterías
6  # FECHA:    2026-06-03
7  # VERSIÓN:  1.0
8  #
9  # DESCRIPCIÓN:
10 #   Monitoriza de forma continua la temperatura del banco de baterías #3.
11 #   Si la temperatura supera 55 °C activa la refrigeración auxiliar,
12 #   desconecta las líneas de carga solar y desvía el consumo del sector
13 #   industrial a la red comercial de respaldo.
14 #   Emite alertas de emergencia si el peligro persiste tras 10 ciclos.
15 #=====
16
17 # — Directivas del sistema —————
18 @modo_seguro;
19 @intervalo(3000);
20 @timeout(600);
21
22 # — Inicialización del hardware —————
23 init_grid;
24
25 # — Constantes operativas —————
26 declarar CONSTANTE UMBRAL_CRITICO      como DECIMAL := 55.0;
27 declarar CONSTANTE UMBRAL_ADVERTENCIA  como DECIMAL := 45.0;
28 declarar CONSTANTE UMBRAL_SEGURO       como DECIMAL := 40.0;
29 declarar CONSTANTE BATERIA_OBJETIVO    como ENTERO  := 3;
30 declarar CONSTANTE MAX_CICLOS_EMERGENCIA como ENTERO  := 10;
31 declarar CONSTANTE SECTOR_INDUSTRIAL    como ENTERO  := 5;
32 declarar CONSTANTE PANEL_SOLAR_PRINCIPAL como ENTERO  := 1;
33 declarar CONSTANTE PANEL_SOLAR_SECUNDARIO como ENTERO  := 2;
34
35 # — Variables de estado —————
36 declarar temp_actual      como DECIMAL := 0.0;
37 declarar ciclos_criticos  como ENTERO  := 0;
38 declarar refrigeracion_activa como BOOLEANO := FALSO;
39 declarar carga_solar_desconectada como BOOLEANO := FALSO;
40 declarar sector_desviado   como BOOLEANO := FALSO;
41 declarar sistema_activo    como BOOLEANO := VERDADERO;
42

```

```

43 # — Registro de inicio —————
44 registrar_evento("Inicio de rutina de monitoreo térmico – Batería B-03");
45 mostrar_panel("Sistema de prevención térmica ACTIVO – Monitoreando B-03");
46
47 #=====
48 # BUCLE PRINCIPAL DE MONITOREO CONTINUO
49 #=====
50 mientras sistema_activo ejecutar
51
52     # Lectura del sensor térmico de celda
53     temp_actual := leer_temperatura(BATERIA_OBJETIVO);
54     registrar_evento("Lectura térmica B-03: " + temp_actual + " °C");
55
56     #-----
57     # CASO CRÍTICO: Temperatura supera el umbral de seguridad (> 55 °C)
58     #-----
59     si temp_actual > UMBRAL_CRITICO entonces
60
61         ciclos_criticos := ciclos_criticos + 1;
62         emitir_alerta(3, "CRÍTICO: Temperatura B-03 = " + temp_actual
63         | " °C – Ciclo " + ciclos_criticos);
64
65         # Paso 1: Activar sistemas de refrigeración auxiliar
66         si NO refrigeracion_activa entonces
67             activar_refrigeracion(BATERIA_OBJETIVO);
68             refrigeracion_activa := VERDADERO;
69             registrar_evento("Refrigeración auxiliar ACTIVADA en B-03");
70             mostrar_panel(">> Refrigeración de emergencia ENCENDIDA <<");
71         fin_si
72
73         # Paso 2: Desconectar líneas de carga solar para detener
74         #         el ingreso de energía térmica al sistema
75         si NO carga_solar_desconectada entonces
76             desconectar_carga_solar(PANEL_SOLAR_PRINCIPAL);
77             desconectar_carga_solar(PANEL_SOLAR_SECUNDARIO);
78             carga_solar_desconectada := VERDADERO;
79             registrar_evento("Carga solar DESCONECTADA – Paneles 1 y 2");
80             mostrar_panel(">> Ingreso solar DETENIDO por seguridad <<");
81         fin_si
82

```

```

83 # Paso 3: Desviar el consumo del sector industrial hacia la
84 # red comercial de respaldo para aliviar el estrés
85 si NO sector_desviado entonces
86     conmutar_linea(SECTOR INDUSTRIAL, FALSO);
87     conectar_red_comercial(SECTOR INDUSTRIAL);
88     sector_desviado := VERDADERO;
89     registrar_evento("Sector industrial desviado a red comercial");
90     mostrar_panel(">> Sector 5 operando con RED COMERCIAL <<");
91 fin_si
92
93 # Verificación: Si el peligro persiste tras MAX_CICLOS_EMERGENCIA
94 si ciclos_criticos >= MAX_CICLOS_EMERGENCIA entonces
95     emitir_alerta(4, "EMERGENCIA: Fuga térmica NO contenida en B-03"
96         + " tras " + MAX_CICLOS_EMERGENCIA
97         + " ciclos. Requiere intervención MANUAL.");
98     registrar_evento("ALERTA MÁXIMA: Procedimiento de emergencia activado");
99     mostrar_panel("!!! EVACUAR ZONA DE BATERÍAS – INTERVENCIÓN MANUAL REQUERIDA !!!");
100     sistema_activo := FALSO;
101 fin_si
102
103 #-----
104 # CASO ADVERTENCIA: Temperatura elevada pero no crítica (45-55 °C)
105 #-----
106 sino_si temp_actual > UMBRAL_ADVERTENCIA entonces
107     emitir_alerta(2, "ADVERTENCIA: Temperatura B-03 = "
108         + temp_actual + " °C – Monitoreo intensivo");
109     mostrar_panel("Temperatura elevada en B-03 – Vigilancia activa");
110
111 #-----
112 # CASO RECUPERACIÓN: Temperatura regresa a nivel seguro (< 40 °C)
113 #-----
114 sino_si temp_actual < UMBRAL_SEGURO entonces

```

```

115     si refrigeracion_activa O carga_solar_desconectada O sector_desviado entonces
116         registrar_evento("Temperatura normalizada en B-03: " + temp_actual + " °C");
117         emitir_alerta(1, "INFO: Temperatura B-03 estabilizada. Restaurando operaciones.");
118
119         # Restaurar refrigeración
120         si refrigeracion_activa entonces
121             desactivar_refrigeracion(BATERIA_OBJETIVO);
122             refrigeracion_activa := FALSO;
123             registrar_evento("Refrigeración auxiliar DESACTIVADA");
124         fin_si
125
126         # Restaurar carga solar
127         si carga_solar_desconectada entonces
128             conectar_carga_solar(PANEL_SOLAR_PRINCIPAL);
129             conectar_carga_solar(PANEL_SOLAR_SECUNDARIO);
130             carga_solar_desconectada := FALSO;
131             registrar_evento("Carga solar RECONECTADA – Paneles 1 y 2");
132         fin_si
133
134         # Restaurar sector industrial a la microred
135         si sector_desviado entonces
136             desconectar_red_comercial(SECTOR INDUSTRIAL);
137             conmutar_linea(SECTOR INDUSTRIAL, VERDADERO);
138             sector_desviado := FALSO;
139             registrar_evento("Sector industrial restaurado a microred local");
140         fin_si
141
142         ciclos_criticos := 0;
143         mostrar_panel("Operaciones normales RESTAURADAS – B-03 estable");
144     fin_si
145
146 #-----
147 # CASO NORMAL: Temperatura dentro de parámetros operativos
148 #-----
149 sino
150     mostrar_panel("B-03 operando normalmente – Temp: " + temp_actual + " °C");
151 fin_si
152
153 fin_mientras
154
155

```

```

156 # — Secuencia de apagado seguro —————
157 registrar_evento("Fin de rutina de monitoreo térmico – Apagado del sistema");
158 apagar_grid;
159

```

Escenario B: Balance de Carga y Optimización Energética Autónoma

Este script evalúa dinámicamente las variables agregadas de la planta. Si el almacenamiento es óptimo y la generación sobrepasa la demanda local, habilita la inyección comercial hacia la red pública externa. En contraposición, si detecta niveles críticos de carga nocturna, ejecuta un algoritmo iterativo de corte selectivo para aislar sectores no esenciales, salvaguardando líneas vitales como infraestructura de servidores y sistemas médicos.

```
1  #=====
2  # PROGRAMA ECO-L – Escenario Operativo B
3  # TÍTULO: Balance de Carga y Optimización Energética Autónoma
4  # PLANTA: ECO-GRID – Centro de Control de Distribución
5  # AUTOR: Operador Senior - Optimización Energética
6  # FECHA: 2026-06-03
7  # VERSIÓN: 1.0
8  #
9  # DESCRIPCIÓN:
10 # Evalúa el estado de carga de los bancos de baterías.
11 # Si la carga es óptima (> 90%) y la generación solar excede la
12 # demanda, inyecta el excedente a la red pública.
13 # Si la carga es crítica (< 20%) en horario nocturno, aísla los
14 # sectores no esenciales para preservar el suministro en áreas
15 # médicas y de servidores críticos.
16 #=====
17
18 # — Directivas del sistema —————
19 @modo_seguro;
20 @intervalo(5000);
21 @timeout(3600);
22
23 # — Inicialización del hardware —————
24 init_grid;
25
26 # — Constantes de configuración —————
27 declarar CONSTANTE CARGA_OPTIMA como DECIMAL := 90.0;
28 declarar CONSTANTE CARGA_CRITICA como DECIMAL := 20.0;
29 declarar CONSTANTE CARGA_RECUPERACION como DECIMAL := 35.0;
30 declarar CONSTANTE HORA_INICIO_NOCTURNO como ENTERO := 18;
31 declarar CONSTANTE HORA_FIN_NOCTURNO como ENTERO := 6;
32 declarar CONSTANTE EXCEDENTE_MINIMO_KW como DECIMAL := 5.0;
33
34 # — Identificadores de dispositivos —————
35 declarar CONSTANTE BATERIA_PRINCIPAL como ENTERO := 1;
36 declarar CONSTANTE BATERIA_RESPALDO como ENTERO := 2;
37 declarar CONSTANTE FUENTE_SOLAR como ENTERO := 1;
38 declarar CONSTANTE FUENTE_EOLICA como ENTERO := 2;
39
```

```

40 # — Identificadores de sectores de consumo —————
41 declarar CONSTANTE SECTOR_PRODUCCION      como ENTERO := 1;
42 declarar CONSTANTE SECTOR_ILUMINACION     como ENTERO := 2;
43 declarar CONSTANTE SECTOR_CLIMATIZACION   como ENTERO := 3;
44 declarar CONSTANTE SECTOR_MEDICO          como ENTERO := 4;
45 declarar CONSTANTE SECTOR_SERVIDORES      como ENTERO := 5;
46 declarar CONSTANTE SECTOR_CAFETERIA       como ENTERO := 6;
47 declarar CONSTANTE SECTOR_OFICINAS        como ENTERO := 7;
48
49 # — Variables de estado operativo —————
50 declarar carga_bateria_ppal  como DECIMAL := 0.0;
51 declarar carga_bateria_resp  como DECIMAL := 0.0;
52 declarar generacion_solar    como DECIMAL := 0.0;
53 declarar generacion_eolica   como DECIMAL := 0.0;
54 declarar generacion_total     como DECIMAL := 0.0;
55 declarar demanda_total       como DECIMAL := 0.0;
56 declarar excedente_kw         como DECIMAL := 0.0;
57 declarar hora_actual          como ENTERO  := 0;
58 declarar es_horario_nocturno como BOOLEANO := FALSO;
59 declarar sectores aislados    como BOOLEANO := FALSO;
60 declarar inyeccion_activa     como BOOLEANO := FALSO;
61 declarar sistema_operativo    como BOOLEANO := VERDADERO;
62
63 # — Registro de inicio —————
64 registrar_evento("Inicio de rutina de balance y optimización energética");
65 mostrar_panel("Sistema de optimización ECO-GRID ACTIVO");
66
67 #=====
68 # BUCLE PRINCIPAL DE BALANCE ENERGÉTICO
69 #=====
70 mientras sistema_operativo ejecutar
71
72     # — Fase 1: Recolección de datos de sensores —————
73     carga_bateria_ppal := estado_carga(BATERIA_PRINCIPAL);
74     carga_bateria_resp := estado_carga(BATERIA_RESPALDO);
75     generacion_solar   := leer_generacion(FUENTE_SOLAR);
76     generacion_eolica  := leer_generacion(FUENTE_EOLICA);
77     generacion_total   := generacion_solar + generacion_eolica;
78     hora_actual        := obtener_hora();
79

```

```

80 # Calcular demanda total sumando todos los sectores activos
81 demanda_total := 0.0;
82 para sector_id desde 1 hasta 7 ejecutar
83 |   demanda_total := demanda_total + leer_demanda(sector_id);
84 fin_para
85
86 # Determinar si estamos en horario nocturno (18:00 - 06:00)
87 si hora_actual >= HORA_INICIO_NOCTURNO O hora_actual < HORA_FIN_NOCTURNO entonces
88 |   es_horario_nocturno := VERDADERO;
89 sino
90 |   es_horario_nocturno := FALSO;
91 fin_si
92
93 registrar_evento("Estado: Carga=" + carga_bateria_ppal
94 |               | + "% | Gen=" + generacion_total
95 |               | + " kW | Dem=" + demanda_total
96 |               | + " kW | Hora=" + hora_actual);
97
98 #=====
99 # CASO A: Carga óptima + generación excede demanda → INYECTAR
100 #=====
101 si carga_bateria_ppal > CARGA_OPTIMA Y generacion_total > demanda_total entonces
102
103     excedente_kw := generacion_total - demanda_total;
104     mostrar_panel("Excedente detectado: " + excedente_kw + " kW disponibles");
105
106     # Verificar que el excedente justifica la inyección (mínimo 5 kW)
107     si excedente_kw > EXCEDENTE_MINIMO_KW entonces
108         inyectar_excedente(excedente_kw);
109         inyeccion_activa := VERDADERO;
110         registrar_evento("INYECCIÓN a red pública: " + excedente_kw + " kW");
111         emitir_alerta(1, "INFO: Inyectando " + excedente_kw
112 |               | + " kW a red pública – Batería al " + carga_bateria_ppal + "%");
113         mostrar_panel(">> VENDIENDO " + excedente_kw + " kW a la red pública <<");
114     sino
115         mostrar_panel("Excedente mínimo (" + excedente_kw + " kW) – Sin inyección");
116     fin_si
117

```

```

118     # Si los sectores estaban aislados, restaurar operaciones normales
119     si sectores_aislados entonces
120         conmutar_linea(SECTOR_CAFETERIA, VERDADERO);
121         conmutar_linea(SECTOR_OFICINAS, VERDADERO);
122         conmutar_linea(SECTOR_CLIMATIZACION, VERDADERO);
123         conmutar_linea(SECTOR_ILUMINACION, VERDADERO);
124         sectores_aislados := FALSO;
125         registrar_evento("Sectores no esenciales RESTAURADOS – Carga recuperada");
126         emitir_alerta(1, "INFO: Todos los sectores reconectados a la microred");
127         mostrar_panel("Sectores no esenciales RESTAURADOS");
128     fin_si
129
130     #=====
131     # CASO B: Carga crítica + horario nocturno → AISLAR sectores
132     #=====
133     sino_si carga_bateria_ppal < CARGA_CRITICA Y es_horario_nocturno entonces
134
135         emitir_alerta(3, "CRÍTICO: Batería principal al " + carga_bateria_ppal
136         |   |   |   | + "% en horario nocturno – Protocolo de aislamiento ACTIVO");
137         registrar_evento("Protocolo de aislamiento nocturno ACTIVADO");
138
139         # Aislar sectores no esenciales para preservar suministro crítico
140         si NO sectores_aislados entonces
141             mostrar_panel(">> AISLANDO sectores no esenciales <<");
142
143             conmutar_linea(SECTOR_CAFETERIA, FALSO);
144             registrar_evento("Sector CAFETERÍA aislado de la microred");
145
146             conmutar_linea(SECTOR_OFICINAS, FALSO);
147             registrar_evento("Sector OFICINAS aislado de la microred");
148
149             conmutar_linea(SECTOR_CLIMATIZACION, FALSO);
150             registrar_evento("Sector CLIMATIZACIÓN aislado de la microred");
151
152             conmutar_linea(SECTOR_ILUMINACION, FALSO);
153             registrar_evento("Sector ILUMINACIÓN NO ESENCIAL aislado");
154
155             sectores_aislados := VERDADERO;
156

```

```

157         # Confirmar protección de sectores críticos
158         emitir_alerta(2, "Sectores críticos PROTEGIDOS: Médico (S4) y Servidores (S5)");
159         mostrar_panel("ÁREAS MÉDICAS y SERVIDORES con suministro GARANTIZADO");
160     fin_si
161
162     # Si la inyección estaba activa, detenerla para conservar energía
163     si inyeccion_activa entonces
164         inyectar_excedente(0.0);
165         inyeccion_activa := FALSO;
166         registrar_evento("Inyección a red pública DETENIDA – Modo conservación");
167     fin_si
168
169     # Verificar si la batería de respaldo puede asistir
170     si carga_bateria_resp > CARGA_CRITICA entonces
171         emitir_alerta(1, "INFO: Batería de respaldo disponible al "
172             + carga_bateria_resp + "%");
173     sino
174         emitir_alerta(4, "EMERGENCIA: Ambas baterías en nivel crítico. "
175             + "Principal=" + carga_bateria_ppal
176             + "% Respaldo=" + carga_bateria_resp + "%");
177         mostrar_panel("!!! ALERTA MÁXIMA: Reserva energética casi agotada !!!");
178     fin_si
179
180     #=====
181     # CASO C: Carga en recuperación – Monitoreo preventivo
182     #=====
183     sino_si carga_bateria_ppal < CARGA_RECUPERACION entonces
184         emitir_alerta(2, "ADVERTENCIA: Batería principal al "
185             + carga_bateria_ppal + "% – Nivel bajo");
186         mostrar_panel("Batería en proceso de carga – Nivel: " + carga_bateria_ppal + "%");
187
188     #=====
189     # CASO D: Operación normal – Reporte de estado
190     #=====
191     sino
192         mostrar_panel("Operación normal | Bat: " + carga_bateria_ppal
193             + "% | Gen: " + generacion_total
194             + " kW | Dem: " + demanda_total + " kW");
195
196     # Restaurar sectores si la carga se recuperó lo suficiente
197     si sectores_aislados Y carga_bateria_ppal > CARGA_RECUPERACION entonces
198         conmutar_linea(SECTOR_CAFETERIA, VERDADERO);
199         conmutar_linea(SECTOR_OFICINAS, VERDADERO);
200         conmutar_linea(SECTOR_CLIMATIZACION, VERDADERO);
201         conmutar_linea(SECTOR_ILUMINACION, VERDADERO);
202         sectores_aislados := FALSO;
203         registrar_evento("Sectores no esenciales restaurados – Carga suficiente");
204         mostrar_panel("Todos los sectores RECONECTADOS a la microrred");
205     fin_si
206 fin_si
207
208 fin_mientras
209
210 # — Secuencia de apagado seguro —————
211 registrar_evento("Fin de rutina de balance y optimización – Apagado del sistema");
212 mostrar_panel("Sistema de optimización ECO-GRID FINALIZADO");
213 apagar_grid;
214

```

El desarrollo integral del Lenguaje L (ECO-L) demuestra de manera fehaciente cómo la correcta aplicación de las fases del análisis léxico y sintáctico permite estructurar un software robusto, predecible y altamente especializado para entornos de misión crítica. El uso coordinado de una gramática BNF libre de ambigüedades garantiza la correcta traducción de las primitivas industriales, proveyendo a los operadores una herramienta fiable y de muy baja tasa de error humano.

Conclusiones

El análisis integral de este estudio permite concluir que la arquitectura de un lenguaje de programación no es un elemento aislado, sino el resultado directo de una vinculación profunda entre la teoría gramatical y la eficiencia operativa. A través del benchmarking realizado con la Conjetura de Collatz, se evidenció que los niveles morfológicos y sintácticos determinan el modelo de ejecución: mientras que la flexibilidad de Python introduce una latencia significativa debido a su interpretación por bytecode (11,031.53 ms), la rigurosidad léxica y el tipado estricto de Zig permiten una traducción directa a código máquina, logrando una predictibilidad y consumo de memoria (< 1 MB) indispensables para sistemas de bajo nivel.

Asimismo, las observaciones prácticas demostraron que la abstracción sintáctica juega un papel dual. En lenguajes de propósito general como JavaScript, el motor V8 utiliza la compilación Just-In-Time (JIT) para transformar una sintaxis dinámica en instrucciones altamente optimizadas en tiempo real (38.17 ms), equilibrando la facilidad de desarrollo con un rendimiento comparable al nativo. Esta capacidad de optimización subraya que la eficiencia estructural del software depende tanto del diseño del lenguaje como de la infraestructura tecnológica que lo procesa.

Finalmente, el diseño del DSL ECO-L validó que la aplicación estricta de gramáticas formales (EBNF) y alfabetos bien definidos es la ruta más efectiva para garantizar la seguridad en entornos de misión crítica. La abstracción hacia un "español estructurado" en ECO-L no es solo una mejora de legibilidad, sino una estrategia de seguridad por diseño: al restringir el dominio semántico y prohibir estructuras ambiguas o peligrosas (como los punteros), se logra una traducción técnica unívoca hacia los actuadores de la planta. En definitiva, para el ingeniero informático moderno, la maestría en los mecanismos de traducción y ejecución es lo que permite transformar una abstracción lógica en una solución industrial robusta, eficiente y determinista.

Bibliografía

- Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). Compiladores: Principios, técnicas y herramientas (2da ed.). México: Pearson Educación.
-
- Derivando. (21 de octubre de 2015). La conjetura de Collatz [Archivo de video]. Recuperado de https://youtu.be/HpcYW08Ug7g?si=Wy-trWLRXosYs_te
- Fowler, M. (2010). Domain-Specific Languages. Boston, MA: Addison-Wesley Professional.
- GeoMaths. (23 de junio de 2024). Collatz Conjecture. Recuperado de <https://geomaths.co.uk/2024/06/23/collatz-conjecture/>
- Louden, K. C. (2004). Construcción de compiladores: principios y práctica. México: Thomson.
- MID UDEV. (2025). Curso completo de python desde cero 2025 [Archivo de video]. Recuperado de https://youtu.be/TkN2i-_4N4g?si=6CWb6zpCBaqUHIgg
- Parr, T. (2009). Language Implementation Patterns: Create Your Own Domain-Specific and General-Purpose Languages. Raleigh, NC: Pragmatic Bookshelf.
- Zig Software Foundation. (2026). Descargas de Zig. Recuperado de <https://ziglang.org/es-AR/download/>